

Docker Interview Preparation

18 Essential Questions & Answers for DevOps Engineers

Q1. How does Docker architecture work?

Docker uses a **client-server architecture** with three main components:

- **Docker Client** – CLI tool (docker) that sends commands to the daemon via REST API.
- **Docker Daemon (dockerd)** – Background service that manages images, containers, networks, and volumes.
- **Docker Registry** – Stores images (Docker Hub is the default public registry; ECR/GCR for private use).

When you run **docker run nginx**, the client sends the request to the daemon, which pulls the image from the registry (if not cached), creates a container, and starts it. The daemon is the brain — the client is just a thin interface.

Q2. What is the difference between a Docker image and a container?

Image = Blueprint (read-only). A layered filesystem snapshot built from a Dockerfile. It does not run or consume CPU/RAM.

Container = Running instance of an image. It has a writable layer on top, its own PID namespace, network, and filesystem. You can run many containers from the same image.

Analogy: Image is a class definition; container is an instantiated object.

Q3. What are Docker layers and why are they important?

Every instruction in a Dockerfile (RUN, COPY, ADD) creates a new **read-only layer**. Layers are stacked — Docker uses a Union filesystem (OverlayFS) to merge them into a single view.

Why they matter:

- **Cache reuse** – Unchanged layers are cached. Rebuild only what changed.
- **Storage efficiency** – Layers shared across images are stored once on disk.
- **Faster CI builds** – Order your Dockerfile so rarely-changing layers (apt installs) come before frequently-changing ones (COPY . .).

Q4. What is a Dockerfile and how do you write one?

A Dockerfile is a text file with instructions to build a Docker image. Each instruction becomes a layer.

Example — Node.js app:

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
```

```
EXPOSE 3000
CMD ["node", "server.js"]
```

- Use alpine base images to keep size small.
- Copy package files first, then source — maximizes cache hits.
- Use .dockerignore to exclude node_modules, .git, etc.

Q5. What is the difference between CMD and ENTRYPOINT?

ENTRYPOINT – Defines the fixed executable that always runs. Cannot be overridden without --entrypoint flag.

CMD – Provides default arguments. Easily overridden at runtime (docker run image my-arg).

Best practice — combine both:

```
ENTRYPOINT ["python", "app.py"]
CMD ["--port", "8080"]
```

Here, python app.py always runs. You can override the port: **docker run myapp --port 9090**

Trap: Using CMD alone means the user can accidentally replace your entire startup command.

Q6. How do you reduce Docker image size?

- **Use minimal base images** – alpine, distroless, or scratch instead of ubuntu.
- **Multi-stage builds** – Build in a fat image, copy only the binary to a clean final stage.
- **Combine RUN commands** – Each RUN is a layer; chain with && and clean up in the same layer.
- **Use .dockerignore** – Exclude test files, docs, .git, node_modules.
- **Remove package manager caches** – e.g., rm -rf /var/lib/apt/lists/* after apt-get install.

Multi-stage example:

```
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

FROM alpine:3.18
COPY --from=builder /app/myapp /usr/local/bin/
ENTRYPOINT ["myapp"]
```

Q7. What is a Docker volume and why is it used?

A **volume** is a Docker-managed storage location outside the container filesystem (stored in /var/lib/docker/volumes/ on the host). Data in volumes **persists** even when the container is deleted.

Use cases:

- Database data (MySQL, PostgreSQL) that must survive container restarts.
- Shared data between multiple containers.
- Avoiding data loss during image upgrades.

```
docker volume create mydata
docker run -v mydata:/var/lib/mysql mysql:8
```

Q8. What is the difference between a bind mount and a volume?

Bind Mount – Maps a specific host path to a container path. You control the location.

Volume – Docker manages the storage location. Portable, easier to back up, works on all platforms.

Key differences:

- Bind mounts depend on host directory structure — not portable across machines.
- Volumes are managed by Docker, isolated from host changes, preferred in production.
- Use bind mounts for local development (live code reloading). Use volumes in production.

```
# Bind mount
docker run -v /host/path:/container/path nginx

# Volume
docker run -v myvolume:/container/path nginx
```

Q9. How does Docker networking work?

Docker creates a virtual network stack per container. Each container gets its own network namespace, loopback interface, and (depending on network mode) an IP address.

The Docker daemon manages a built-in DNS server — containers on the same user-defined network can reach each other **by container name**.

Flow for a request:

- App calls db:5432 → Docker DNS resolves 'db' to container IP → packet routes via docker0 bridge.
- For external traffic, Docker uses iptables NAT rules to map host ports to container ports (-p 8080:80).

Q10. What are the different Docker network types?

- **bridge** (default) – Containers on same bridge can communicate. Isolated from host. Use for single-host multi-container apps.
- **host** – Container shares the host's network namespace. No isolation. Useful for performance-critical apps.
- **none** – No networking. Completely isolated. Used for batch jobs or security-sensitive workloads.
- **overlay** – Spans multiple Docker hosts. Used in Docker Swarm or multi-host setups.
- **macvlan** – Assigns a real MAC address to the container. Makes it appear as a physical device on the network.

Interview tip: Always prefer user-defined bridge networks over the default bridge — they support DNS resolution by name.

Q11. How do you debug a failed container?

Step-by-step approach:

- Check exit code: **docker inspect <container> --format '{{.State.ExitCode}}'**
- Read logs: **docker logs <container> --tail 100**
- If container won't start, override entrypoint: **docker run --entrypoint sh myimage**
- Exec into running container: **docker exec -it <container> /bin/sh**
- Inspect full metadata: **docker inspect <container>** — check Env, Mounts, NetworkSettings.
- Check resource limits: **docker stats <container>** — OOMKill = memory limit exceeded.

Exit code 137 = OOM killed. **Exit code 1** = app crashed. **Exit code 127** = command not found.

Q12. How do you check container logs?

```
docker logs <container_name>
docker logs -f <container>      # Follow live
docker logs --tail 50 <container> # Last 50 lines
docker logs --since 1h <container> # Last 1 hour
```

In production, configure a log driver (json-file, fluentd, awslogs, gelf) in daemon.json or per-container so logs are shipped to a centralized system like ELK or CloudWatch — not stored on the host.

Q13. How do you handle container restart policies?

- **no** (default) – Never restart.
- **always** – Always restart, even on Docker daemon restart. Use for critical services.
- **unless-stopped** – Restart always except when manually stopped. Best for most production containers.
- **on-failure[:max-retries]** – Restart only on non-zero exit. Optional retry limit.

```
docker run --restart=unless-stopped nginx
docker run --restart=on-failure:3 myapp
```

Caution: 'always' with a crashing container causes rapid restart loops. Monitor with docker stats or set on-failure with a retry limit.

Q14. What is Docker Compose and when do you use it?

Docker Compose is a tool for defining and running **multi-container applications** using a YAML file (docker-compose.yml). One command starts all services, networks, and volumes together.

Use when:

- Local development with multiple services (app + DB + Redis + queue).
- Integration testing in CI pipelines.
- Small-scale deployments that don't need Kubernetes overhead.

```
services:
  app:
    build: .
    ports: ["8080:8080"]
    depends_on: [db]
  db:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Q15. What is the difference between Docker Compose and Docker Swarm?

Docker Compose – Single-host orchestration. Designed for development and simple deployments. No high availability.

Docker Swarm – Multi-host cluster orchestration. Adds service scaling, load balancing, rolling updates, and self-healing across nodes.

Key differences:

- Swarm spans multiple machines; Compose does not.
- Swarm has built-in service discovery and load balancing across replicas.
- Compose files can be deployed to Swarm using 'docker stack deploy'.

Reality check: In modern DevOps, Kubernetes has largely replaced Swarm for production multi-host orchestration. Swarm is simpler but has less ecosystem support.

Q16. How do you secure Docker containers?

- **Run as non-root** – Add USER in Dockerfile. Never run as UID 0 in production.
- **Read-only filesystem** – docker run --read-only. Mount only what needs to be writable.
- **Drop Linux capabilities** – docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE.
- **Scan images** – Use Trivy or Snyk in CI: trivy image myapp:latest.
- **Don't store secrets in images** – Use Docker secrets, AWS Secrets Manager, or Vault.
- **Use distroless/scratch images** – Minimal attack surface, no shell.
- **Enable Docker Content Trust** – DOCKER_CONTENT_TRUST=1 for signed images.
- **Limit resources** – --memory=512m --cpus=1 to prevent container escape via resource exhaustion.

Q17. What are common Docker issues in production?

- **Container OOM killed** – Set memory limits and monitor with docker stats. Exit code 137.
- **Disk full from dangling images/volumes** – Run docker system prune -f regularly or automate it.
- **Port conflicts** – Two containers binding the same host port. Use dynamic port mapping or a reverse proxy.
- **Slow builds in CI** – Not leveraging layer cache. Fix Dockerfile order and use BuildKit.
- **Secrets in environment variables** – Visible in docker inspect. Use Docker secrets or external vaults.
- **Containers not restarting after node reboot** – Missing --restart policy or not using a process supervisor.
- **Networking issues between containers** – Using default bridge (no DNS). Switch to user-defined networks.

Q18. What are best practices for Docker in production?

- **Use specific image tags** – Never use :latest in production. Pin to a digest or semantic version.
- **One process per container** – Don't run nginx + app + cron in one container.
- **Health checks** – HEALTHCHECK in Dockerfile so orchestrators know when a container is ready.
- **Multi-stage builds** – Keep production images lean; dev tooling stays in build stage.
- **Centralized logging** – Use log drivers to ship to ELK/CloudWatch; never rely on docker logs in prod.
- **Resource limits** – Always set --memory and --cpus to prevent noisy-neighbor problems.
- **Image scanning in CI** – Block deploys on HIGH/CRITICAL CVEs with Trivy.
- **Use Docker secrets or external secret managers** – Never bake credentials into images.
- **Immutable containers** – Never modify a running container. Rebuild and redeploy.